

Analysis: Bullseye

First, we need to know the area of the first black ring. It can be calculated by subtracting the area of a white circle with radius r from a black circle with radius $r+1$ cm. That is, the area is $(r+1)^2\pi - r^2\pi$ cm². Expanding we get $(2r+1)\pi$ cm². Since 1mL of paint can cover area π cm², we need exactly $2r+1$ mL of paint to draw the first black ring. For the second black ring, we do the same: subtracting the area of a white circle with radius $r+2$ cm from a black circle with radius $r+3$ cm, thus we need $2r+5$ mL to draw. In general, we need exactly $(r+2k-1)^2 - (r+2k-2)^2 = 2r+4k-3$ mL of paint to print the k -th black ring.

For small we know that the answer is less than $t = 1000$ anyway, so that we can try adding black rings one by one until the total amount of paint used including the next black ring exceeds t , then we stop adding and output the number of black rings we have included so far.

However, this does not work well with the large input as the answer can be much bigger! This can be verified by the fourth sample (we intended to be kind to contestants, but turns out many still failed the large due to integer overflow :-). How can we improve the algorithm?

The key is the following: if you can paint at most k black rings using t mL of paint, for sure you can also use it to draw 1 black ring, 2 black rings, ... up to $k-1$ black rings. So instead of searching for the answers linearly, we can perform [binary search](#) on "Is it possible to use t mL of paint to draw k black rings?" to find the answer more efficiently. Now the remaining question is how much paint is used to draw k black rings.

Looking back about the total amount of paint to draw individual black rings, it is easily observed that they form an arithmetic progression: $2r+1, 2r+5, 2r+9, \dots, 2r+4k-3$. Then the total amount is simply the sum of the progression, which is $(2r+1+2r+4k-3) \times k \div 2 = (2r+2k-1)k$ mL. This is sufficient to completely solve the problem.

Here is a complete solution in Python for reference:

```
num_cases = int(raw_input())
for casenum in range(1, num_cases+1):
    r, t = [int(z) for z in raw_input().split()]
    res, lo, hi = 0, 1, t
    while lo <= hi:
        mid = (lo + hi) / 2
        if mid * (2 * r + 2 * mid - 1) > t:
            hi = mid - 1
        else:
            lo, res = mid + 1, mid
    print "Case #%d: %d" % (casenum, res)
```

You might think that you need big integer libraries to solve the large input, in fact you don't. For example, double precision floating point numbers suffice to check the condition correctly. A more elegant way is to find the first range with **exponential growth** in length so that there exists a value in the range which fails to satisfy the condition. It takes logarithmic time to find the required range. Then we perform binary search on it and compute the answer. Here is a C++ snippet that describes the algorithm without using any big integer library.

```
// Check if we can draw k black rings.
bool Check(long long r, long long t, long long k) {
    return 2 * k * k + (2 * r - 1) * k <= t;
```

```

}

// Find the maximum number of black rings that can be drawn.
long long Solve(long long r, long long t) {
    long long left = 0, right = 1;
    // Find the range that the answer lies in.
    while(Check(r,t,right)) {
        left = right;
        right *= 2;
    }

    // Binary search on the range [left, right) for the answer.
    while(right - left > 1) {
        long long k = (left + right) / 2;
        if (Check(r, t , k))
            left = k;
        else
            right = k;
    }
    return left;
}

```

Analysis: Good Luck

The luck factor

This problem is unusual in that you do not have full information and are forced to make guesses as part of the solution. We thought it would be a fun change from the usual deterministic setting.

Nevertheless, luck did not play a huge role in this problem. The first dataset was easy enough for many approaches to work. The second dataset was harder, but we estimated that an optimal solution would have very good chances: the probability of an optimal solution failing is only on the order of 1 in a million! This is because 8000 is a lot of independent guesses, and the limit **X** is rather conservative: about 5 standard deviations below the expected number of correct guesses.

The optimal strategy

One may be tempted to apply various heuristics to try to reason about what kind of hidden numbers are likely. In this case, it is best to approach the problem scientifically and simply always go for the highest probability of success!

In order to do that, we compute the probability of each of the 18564 possibilities (for the larger dataset) and pick the largest one.

Why 18564? There are 7 choices for each of the 12 hidden numbers. That seems to give $7^{12} = 13841287201$ possibilities, which is a lot. But, the order of hidden numbers doesn't matter, which reduced the number of different possibilities to $(12+7-1) \text{ choose } (7-1) = 18564$. Try to derive this formula! Or just generate them all and count.

A priori probabilities: **K=0**

What if **K=0**, so we have no information about the hidden numbers at all? It may seem like then it doesn't matter what we guess, since all possibilities are equally likely. Many contestants made this mistake. Some possibilities are more likely a priori than others, even without any additional information!

For example, for the small dataset: 333 is less likely than 234. 6 times less likely, to be exact. Why? Because 234 may have been generated in 6 different ways (234, 243, 324, 342, 423, 432) while 333 can be generated in only 1 way.

In general, if digit **d** appears **C_d** times among hidden cards, the probability of that set is $\mathbf{N!} / (\mathbf{C_2!} * \dots * \mathbf{C_M!} * (\mathbf{M-1})^{\mathbf{N}})$.

K=1 and Bayes' theorem

So we have computed the a priori probability of every set of hidden cards, but that does not use the crucial available information: the **K** products of random subsets. How do we use that information? Conditional probabilities are the right tool for the job.

Let's start with $K=1$. For each set of hidden numbers, $\mathbf{A}$, we already know the probability of that set happening, $\Pr(\mathbf{A})$. We also know a product $\mathbf{p}$ of a random subset of these numbers. What we are trying to compute is the conditional probability that the hidden set is $\mathbf{A}$ given that the product of a random subset is $\mathbf{p}$. Let's write that as $\Pr(\mathbf{A} \mid \mathbf{p})$.

How to compute that? Use the definition of conditional probabilities:

$$\Pr(\mathbf{A} \mid \mathbf{p}) = \Pr(\mathbf{A} \cap \mathbf{p}) / \Pr(\mathbf{p}) = \Pr(\mathbf{A}) * \Pr(\mathbf{p} \mid \mathbf{A}) / \Pr(\mathbf{p})$$

This derivation is called Bayes' theorem.

We already know $\Pr(\mathbf{A})$, so we only need to know $\Pr(\mathbf{p} \mid \mathbf{A})$. We can pre-compute these values for every $\mathbf{A}$. Simply try every possible subset of each possible set of hidden numbers, see what the products are in each case, and build a large table of all these probabilities. There are $18564 * 2^{12} \approx 76$ million such subsets.

$\Pr(\mathbf{p})$ can then be computed as the sum of $\Pr(\mathbf{A}) * \Pr(\mathbf{p} \mid \mathbf{A})$ over all $\mathbf{A}$.

The complete solution

K is greater than 1, but that's not a problem: we iterate the above reasoning for each of the K products, adjusting the probabilities of the hidden combinations in the process.

The full solution is then:

- Some precomputation:
 - Generate all possible combinations of hidden numbers, ignoring order.
 - Compute the initial probability of each of these hidden sets.
 - For each possible hidden set, find all possible products of subsets and compute $\Pr(\mathbf{p} \mid \mathbf{A})$. Index these values by $\mathbf{p}$ for easy lookup later.
- For each hidden set:
 - Start with the pre-computed initial probability distribution over possible hidden sets.
 - Read one product $\mathbf{p}$ at a time, and adjust the probability distribution by using Bayes' theorem and the pre-computed conditional probabilities $\Pr(\mathbf{p} \mid \mathbf{A})$.
 - Output the most probable possibility.

Analysis: Manage your Energy

The small dataset

The limits in the small dataset are very small for this problem, and they allow brute-force approaches. With a fast enough language, even an algorithm that for each activity in the row tries each possible amount of energy usage (from 0 to current energy) will run in time.

The large dataset obviously does not allow such a solution — both the number of activities and the number of possible amounts of energy to consider are significantly too large.

The highest-valued activity

We will begin solving this problem by looking at the highest-value activity (if there is more than one, pick any of them), let this be activity number **a**. We will prove that there exists an optimal solution that uses full **E** joules for this activity.

Consider any optimal solution. First assume that we had less than **E** joules at the start of activity **a**. This means we spent energy on some other activity before (since we started with **E** energy), let **b** be the number of the last activity we spent non-zero energy on before **a**. Now consider a solution in which we spend one joule less on **b**. Since our energy wasn't **E** in the original solution at the beginning of **a**, if we spend one less joule on **b**, we will still have at most **E** energy at the beginning of **a**, and so it won't go to waste. Thus, we can spend this extra joule on **a**, and then proceed as in the original solution, since after **a** we have exactly the same amount of energy as in the original. The difference in values of these two solutions is $-v_b + v_a$, which is non-negative (since **a** was one of the highest-value solutions). We can repeat this procedure until we obtain a solution which is no worse than the original (and thus still optimal), and has **E** joules at the beginning of **a**.

Now assume that we have an optimal solution that enters **a** with **E** joules, but does not use all of them on **a**. We can change it in a similar fashion. Find the first activity after **a** on which we spend non-zero energy, call it **c**. If it doesn't exist, or if any energy is wasted between **a** and **c**, we can simply expend one more joule on **a** and obtain a strictly better solution. Otherwise, we can spend one joule less on **c** and one more on **a**, to obtain a solution that's no worse.

After repeating these procedures as long as we can, we obtain an optimal solution in which, indeed, we spend full **E** joules on **a**.

Other activities

Notice that the fact we are spending **E** energy on **a** has some implications on the nearby activities. For instance, we will have at most **R** energy available at **a+1**, at most **2R** at **a+2**, etc. At the same time, we need to leave **a-1** with at least **E-R** energy, leave **a-2** with at least **E-2R** energy, and so on.

Consider the activity **d** with the next-highest value after **a**. We have a limit on how much energy we have at most when entering **d** (it's the lower of two numbers — **E** and whatever limit the spending on **a** imposed); and we know how much energy we have left unspent (the higher of 0 and the limit imposed by **a**). We will spend anything between these two limits on activity **d** — a reasoning identical to the one for **a** proves that this doesn't prevent us from getting an optimal solution.

We will continue in this fashion. At each step, we will have for each activity an amount of energy that we have to leave after ending it for activities we already considered, and the maximum amount of energy we can have beginning this activity. At each step, we will take the highest-valued activity we haven't considered yet, and assign to it all the energy we can, updating the limits afterwards.

The time complexity of this solution, as formulated, is $O(N^2)$ — in each of the N steps we find the highest-valued activity not considered yet, assign energy to it, and update limits for all other activities. Due to the constraints on input size that we have this will be fast enough (at least if written reasonably efficiently). It is possible to do better, though.

An $O(N \log N)$ solution

First note that finding the highest-valued activity not considered yet can be done faster — it's enough to just sort the activities by value up front, and then consider them in descending order.

The more difficult part is updating the limits. To do this, we look at how the limits are set again. Each activity we assign imposes a limit on how much energy do we have in later activities, and how much do we have to leave behind in the previous activities. We would like to prove that for each activity a the limit we have to consider comes from the latest activity in the day we considered before a for how much energy we have, and the first activity we considered later in the day than a for how much we need to leave unspent.

This is not surprising. Consider, for instance, the activity b , which is the latest activity considered before a , and some activity c which comes even earlier in the day. If we considered c before b , the loss of energy on activity c is already taken into account when we consider b , so the limits imposed by b will be no less stringent. On the other hand, if we considered c after b , it means that the energy spent on c was already limited so that it would all be recovered by the time we reach b — and so it does not impact the amount of energy we have available at a . A similar reasoning works for activities later in the day than a .

A solution in which each step takes logarithmic time will take each considered activity and insert it (along with the limits it imposes) into a binary search tree allowing logarithmic-time insertions and lower/upper-bound operations (like the "set" structure of many languages). Then, at each subsequent activity, we find the nearest activity already considered before and after it, compute the limits they impose on the current activity, spend all the energy we can (updating the total value) and insert this activity (along with new limits) into the tree.

Note that if $R \geq E$, you can just assume $R = E$, since any energy you regain above E will necessarily go to waste. We observed this and changed the problem statement to avoid this unnecessary case; but we missed some of the inputs in which this should have been corrected, due to which we had to change the statement back and issue a clarification during the contest. We apologize for the confusion this caused.

An $O(N)$ solution

One nice thing about running a contest for a group of very smart people is that they come up with solutions for problems that the authors didn't even think of! This was the case with this problem — after the contest we learned some contestants came up with a $O(N)$ solution.

The key to this solution is observing that we can actually know up front how much energy we will want to spend at a given day. As seen above, if there is no more valuable activity ahead of us, we should just spend all the energy we have at the moment. On the other hand, if there is something more valuable, it's always a good idea to save up energy if we will be able to use it

for the more valuable activity. To make use of this we need to know, for each activity, the nearest activity in the future that's more valuable.

Calculating such an array is a classic problem, it is described, e.g., as the "Stock Span Problem" in the [Wikipedia article on stacks](#). Once we have this array, we can, for each activity in chronological order, do the following:

- If there is no more valuable activity coming up, spend all the energy we have.
- Consider the nearest more valuable activity **X**. If we can spend any energy, and still have **E** energy when **X** comes (assuming we don't spend any between now and **X**), spend as much as we can while still having **E** when **X** comes. Notice the alternative to spending it now would be to spend it between now and **X** — but there's no activity more valuable than the current one in the period to make it worthwhile.
- If we can't spend any energy and still have **E** when **X** comes, we shouldn't spend any — all our energy will be better spent at **X**.

Thanks to for pointing this out to us! The credit for this solution goes to pedrosorio; other $O(N)$ solutions are possible as well (see, e.g., misof's submission).